

Positional Embedding in Transformers

Ved Prakash

johnvedprakash@gmail.com

<https://www.linkedin.com/in/ved-prakash-449323117/>

September 21, 2024

1 Introduction

Unlike RNNs or LSTMs, which process input token by token, transformers take the entire sequence (sentence) simultaneously. This is a key characteristic that makes computation faster in the transformer architecture.

Yet, despite their non-sequential processing, transformers still excel at handling sequential data. So, you might be curious: how exactly do transformers capture and preserve the essence of sequence in the data?

2 Positional Encoding

Transformers have a clever mechanism called *positional encoding*. As the name suggests, tokens (words, for simplicity) at each sequence position (sentence, for simplicity) assigned unique positional information. This positional information is a vector of the same dimension as the word embeddings.

Consider the sentence "*The transformer*." Suppose it uses a 2-dimensional word embedding: "The" is embedded as $[0.4, 0.6]$ and "transformer" is embedded as $[0.9, 0.7]$. The positional embeddings will also be 2-dimensional vectors. For the first position, assume it is $[0, 1]$ and for the second position, it is $[0.84, 0.54]$.

In the transformer architecture, the "positional embedding" is added to the "word embedding" before being passed to the attention layer. Therefore, the vectors that will be passed to the attention layer are as follows:

- For "The": $[0.4 + 0, 0.6 + 1] = [0.4, 1.6]$
- For "transformer": $[0.9 + 0.84, 0.7 + 0.54] = [1.74, 1.24]$

In the attention layer, using learned query and key vectors, the transformer has the ability to understand the sequence and the relationships among the words.

Before you lose attention, let me show some visual graphics here. Figure 1 shows 64-dimensional positional vectors generated for 50 tokens (word) long

sequence. The X-axis represents the dimension of the vectors and the y-axis represents the token's position (word) in the sequence (sentence).

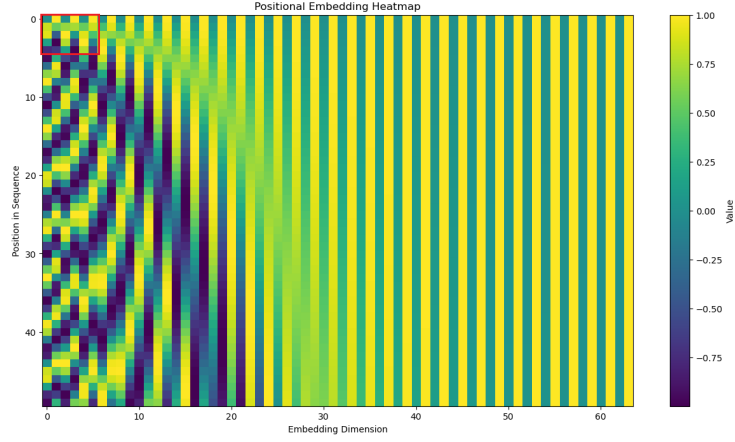


Figure 1: Illustration of 64-dimensional Positional Encoding for 50 sequence

Does figure 1 look confusing? Let's zoom at the first 5 rows and first 6 columns, i.e., the first 6 dimensions of the first 5 sequences. It is bordered by a red rectangle in figure 1.

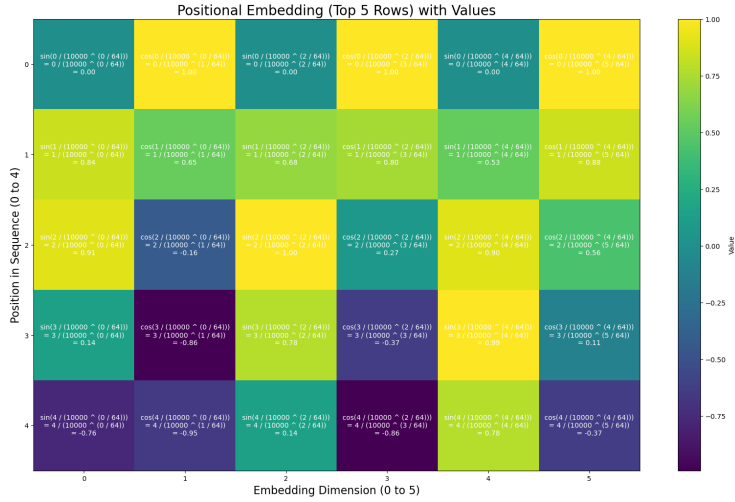


Figure 2: Illustration of Positional Encoding: Zooming at first 5 position and first 6 dimensions

In each cell, specific calculations and values have been provided. These calculations illustrate how the transformer architecture computes the positional

vector for each position. The formula used for this calculation is explained in the subsequent section.

One thing that is clear from figure 1 and 2 is that each position in a sequence will have a unique vector embedding. Additionally, for all sequences (sentences), the same position will correspond to the same positional vector if the dimensions of the vectors are the same.

Now, after getting clear understanding of the 1 and 2, let's reverse the axes, representing the sequence on the X-axis and the dimensions of the embedding vector on the Y-axis.

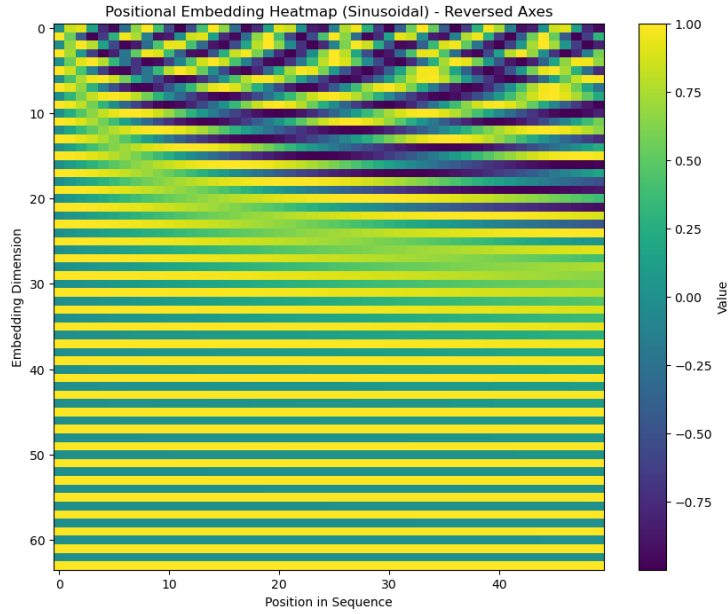


Figure 3: Illustration of Positional Encoding Sequence on X-Axis and Embedding dimension on Y-axis

In figure 3, it can be seen that the lower dimension of the embedding vector changes more than the higher dimension.

Let's visualize the same by plotting individual sin and cosine plots for each dimension.

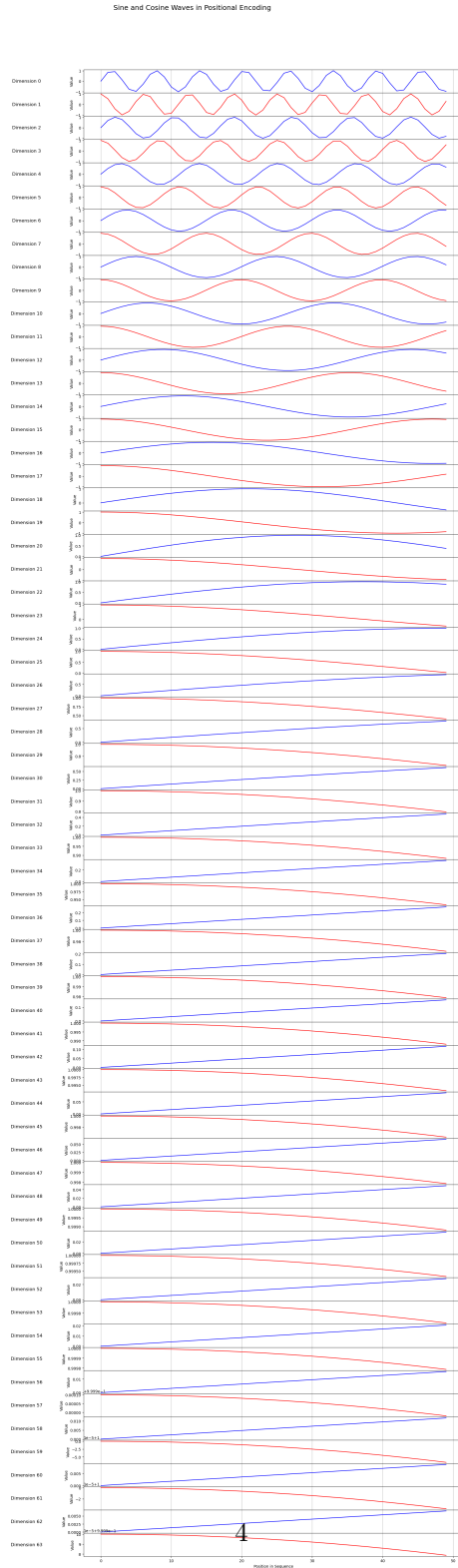


Figure 4: Sine and Cosine wave for positional embedding

From figure 3 and 4, it is clear that the lower dimension of the embedding vector has more frequency than the higher dimension. This means that the positional information encoded in the lower dimensions oscillates more rapidly, allowing the model to capture fine-grained positional relationships between tokens. In contrast, the higher dimensions represent slower oscillations, which can capture broader positional contexts. This frequency distinction enables the model to effectively differentiate between nearby and distant tokens, thereby enhancing its ability to understand context and relationships within the input sequence.

I hope, now positional encoding is clear. Just to give more clues about how it represents each position so uniquely, just get reminded about the binary representation of numbers.

Number	Bit 0	Bit 1	Bit 2
0	0	0	0
1	1	0	0
2	0	1	0
3	1	1	0
4	0	0	1
5	1	0	1
6	0	1	1
7	1	1	1

Table 1: Binary Representation of Numbers from 0 to 7

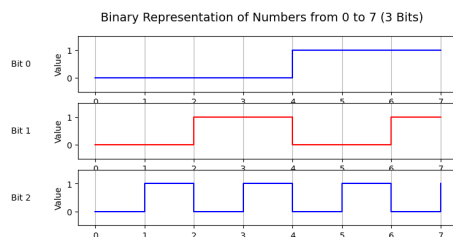


Figure 5: Binary wave Representation of Numbers from 0 to 7

Note: If you're not interested in the mathematics behind this, you can stop here. Up to this point, I believe the conceptual understanding of positional embedding in transformers has been adequately explained. If you wish to delve into the mathematical details, please proceed to the subsequent sections.

3 Fixed Positional Encoding in Transformer Architecture

Positional information can either be learned or fixed a priori. The authors of the paper “*Attention is All You Need*” experimented with both methods and found that the two versions produced nearly identical results. However, for their use case, they opted for *fixed* positional encodings (based on certain mathematical formulas, e.g., sinusoidal functions), as this may allow the model to generalize to sequence lengths longer than those encountered during training.

Now let’s go deeper into the fixed positional encoding scheme, which is based on sine and cosine functions. The key idea is to assign each position pos in a sequence a unique encoding vector. This vector contains values generated from sinusoidal functions of varying frequencies.

The positional encoding for a token at position pos in a sequence is defined as:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$
$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

where:

- $PE(pos, 2i)$: represents the positional encoding at position pos for even indices of the embedding dimension (the $2i$ -th dimension).
- $PE(pos, 2i + 1)$: represents the positional encoding for odd indices of the embedding dimension (the $2i + 1$ -th dimension).
- pos : is the position of the token in the input sequence.
- d_{model} : is the dimension of the model’s embedding space (i.e., the number of features in the embedding vector for each token).
- i : is an abstract index used to calculate the positional encoding for pairs of dimensions in the embedding vector. The actual dimensions of the embedding vector are $2i$ (even) and $2i + 1$ (odd).

3.1 Explanation of the Formula

Let’s break down what the formula is doing:

- The position pos is divided by $10000^{\frac{2i}{d_{\text{model}}}}$. The factor $10000^{\frac{2i}{d_{\text{model}}}}$ is used to scale the input pos differently for each dimension i of the positional encoding. This ensures that the sine and cosine functions vary at different frequencies for different dimensions of the positional encoding.

- At $i = 0$, which corresponds to the 0th (even) and 1st (odd) dimensions of the embedding vector, the positional encoding is as follows:

$$PE(pos, 0) = \sin\left(\frac{pos}{10000^0/d_{\text{model}}}\right) = \sin(pos)$$

$$PE(pos, 1) = \cos\left(\frac{pos}{10000^0/d_{\text{model}}}\right) = \cos(pos)$$

Here, the encoding is dominated by the position pos itself because the denominator is 1. This means that for the first two dimensions, the sine and cosine functions directly encode the position with no scaling.

- If the position $pos = 0$, the values of the sine and cosine functions are:

$$\sin(0) = 0 \quad \text{and} \quad \cos(0) = 1$$

This means that when the token is at position 0 in the sequence, the positional encoding for the 0th and 1st dimensions will be:

$$PE(0, 0) = 0 \quad \text{and} \quad PE(0, 1) = 1$$

This provides a simple and intuitive baseline encoding at the start of the sequence.

- At larger values of i , corresponding to later dimensions, the factor $10000^{\frac{2i}{d_{\text{model}}}}$ grows larger, leading to slower oscillations in the sine and cosine functions. This creates a positional encoding that changes more gradually across dimensions as i increases. For example, at higher dimensions:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

and

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

Here, the values of the sine and cosine functions oscillate more slowly, providing the model with a way to encode relationships at longer ranges between positions in the sequence.

- The use of sine for even dimensions and cosine for odd dimensions ensures that the encoding is periodic, meaning that tokens at similar relative positions in different sequences will have similar encodings. This periodicity helps capture the relative positioning of tokens across the sequence.
- The value 10000 is a hyperparameter that controls the frequency of the sine and cosine functions. Larger values of 10000 result in slower oscillations, while smaller values would produce faster oscillations. In the context of positional encoding, the choice of 10000 ensures that the encoding values change smoothly but in a meaningful way for sequences of varying lengths.

3.2 Why Use Sine and Cosine?

The choice of sine and cosine functions is not arbitrary. These periodic functions provide a smooth and continuous representation of position, and their unique frequency patterns for each dimension allow the model to differentiate between positions. Additionally, sine and cosine functions allow the model to extrapolate positions it has not seen during training. For example, the model can still understand the relationships between tokens even when the sequence length during inference is longer than during training.

The encoding is such that positions close to each other in the sequence will have similar encodings, while positions farther apart will have more distinct encodings. This property makes it easier for the transformer to model relationships between tokens based on their positions in the sequence.

3.3 Illustrative Example

To give a clearer picture, let's say we have a token at position $pos = 5$, and we are working with an embedding dimension of $d_{\text{model}} = 512$. For the first dimension ($i = 0$, the 0th dimension in the encoding), the positional encoding would be:

$$PE(5, 0) = \sin\left(\frac{5}{10000^{\frac{0}{512}}}\right) = \sin(5)$$

For the second dimension ($i = 1$, the 1st dimension in the encoding):

$$PE(5, 1) = \cos\left(\frac{5}{10000^{\frac{1}{512}}}\right)$$

As i increases, the denominator inside the sine and cosine functions grows larger, leading to slower oscillations in the encoding values. This scaling mechanism ensures that each dimension captures positional information at different frequencies, which in turn enables the model to handle sequences of varying lengths and complexities.

4 Relative Positional Information

In the "attention is all you need" paper, authors have mentioned that the positional encoding (PE) PE_{pos+k} can be represented as a linear function of PE_{pos} . Let's understand that in detail and prove it mathematically.

Relative positional encoding focuses on the relationships between tokens rather than their absolute positions. This is particularly useful in tasks where proximity influences meaning.

Besides capturing absolute positional information, the above positional encoding also allows a model to easily learn to attend by relative positions. This is because for any fixed position offset δ , the positional encoding at position $i + \delta$ can be represented by a linear projection of that at position i :

This projection can be explained mathematically. Denoting $\omega_j = \frac{1}{10000^{2j/d}}$, any pair of $(p_{i,2j}, p_{i,2j+1})$ can be linearly projected to $(p_{i+\delta,2j}, p_{i+\delta,2j+1})$ for any fixed offset δ .

Given two tokens at positions i and j , the relative position $r_{ij} = i - j$ can be represented in a relative positional matrix R :

$$\begin{bmatrix} \cos(\delta w_j) & \sin(\delta w_j) \\ -\sin(\delta w_j) & \cos(\delta w_j) \end{bmatrix}$$

The matrix above is a rotation matrix, which is commonly used in mathematics to describe how to rotate a point in a two-dimensional space. In this case, the elements of the matrix correspond to the cosine and sine of the angle δw_j , where δ represents the positional offset. The cosine terms determine the horizontal component of the rotation, while the sine terms determine the vertical component.

The reason for the negative sine in the second row is to maintain the proper orientation of the rotated points. In a two-dimensional Cartesian coordinate system, a rotation matrix is structured such that:

- The first row represents the new x-coordinates after rotation.
- The second row represents the new y-coordinates after rotation, with the negative sine ensuring that the rotation is in the counter-clockwise direction.

Thus, when we apply this rotation matrix to the vector $\begin{bmatrix} p_{i,2j} \\ p_{i,2j+1} \end{bmatrix}$, we obtain the new position of the vector as influenced by the relative positional offset.

This leads us to:

$$\begin{aligned} \begin{bmatrix} \cos(\delta w_j) & \sin(\delta w_j) \\ -\sin(\delta w_j) & \cos(\delta w_j) \end{bmatrix} \begin{bmatrix} p_{i,2j} \\ p_{i,2j+1} \end{bmatrix} &= \begin{bmatrix} \cos(\delta w_j) \sin(i w_j) + \sin(\delta w_j) \cos(i w_j) \\ -\sin(\delta w_j) \sin(i w_j) + \cos(\delta w_j) \cos(i w_j) \end{bmatrix} \\ &= \begin{bmatrix} \sin((i + \delta) w_j) \\ \cos((i + \delta) w_j) \end{bmatrix} \\ &= \begin{bmatrix} p_{i+\delta,2j} \\ p_{i+\delta,2j+1} \end{bmatrix} \end{aligned}$$

where the 2×2 projection matrix does not depend on any position index i .

If having difficulty in understanding above equation, let's go back to our high school class derivation in subsection 4.1

4.1 More details on calculation of Relative Positional Information

Suppose, there is an encoding available for token at position i as $\vec{v}_i$. Now, embedding of another token, which is at δ distance from token i , can be derived using linear transformation of $\vec{v}_i$

$$\vec{v}_i = \begin{bmatrix} \sin(t) \\ \cos(t) \\ \sin(\frac{t}{n_1}) \\ \cos(\frac{t}{n_1}) \\ \dots \\ \sin(\frac{t}{n_k}) \\ \cos(\frac{t}{n_k}) \end{bmatrix} \quad \vec{v}_{i+\delta} = \begin{bmatrix} \sin(t+\delta) \\ \cos(t+\delta) \\ \sin(\frac{t+\delta}{n_1}) \\ \cos(\frac{t+\delta}{n_1}) \\ \dots \\ \sin(\frac{t+\delta}{n_k}) \\ \cos(\frac{t+\delta}{n_k}) \end{bmatrix}$$

using below 2 trigonometric addition formula, $\vec{v}_{t+\delta}$ can be expanded further.

$$\sin(\alpha + \beta) = \sin(\alpha) \cos(\beta) + \cos(\alpha) \sin(\beta)$$

$$\cos(\alpha + \beta) = \cos(\alpha) \cos(\beta) - \sin(\alpha) \sin(\beta)$$

$$\vec{v}_{i+\delta} = \begin{bmatrix} \sin(t) \cos(\delta) + \cos(t) \sin(\delta) \\ \cos(t) \cos(\delta) - \sin(t) \sin(\delta) \\ \sin(\frac{t}{n_1}) \cos(\frac{\delta}{n_1}) + \cos(\frac{t}{n_1}) \sin(\frac{\delta}{n_1}) \\ \cos(\frac{t}{n_1}) \cos(\frac{\delta}{n_1}) - \sin(\frac{t}{n_1}) \sin(\frac{\delta}{n_1}) \\ \dots \\ \dots \\ \sin(\frac{t}{n_k}) \cos(\frac{\delta}{n_k}) + \cos(\frac{t}{n_k}) \sin(\frac{\delta}{n_k}) \\ \cos(\frac{t}{n_k}) \cos(\frac{\delta}{n_k}) - \sin(\frac{t}{n_k}) \sin(\frac{\delta}{n_k}) \end{bmatrix}$$

The above vector can be written as multiplication of 2 matrix,

$$\begin{bmatrix} \cos(\delta) & \sin(\delta) & & & & \\ -\sin(\delta) & \cos(\delta) & & & & \\ & & \cos(\frac{\delta}{n_1}) & \sin(\frac{\delta}{n_1}) & & \\ & & -\sin(\frac{\delta}{n_1}) & \cos(\frac{\delta}{n_1}) & & \\ & & & \dots & & \\ & & & \dots & & \\ & & & & \cos(\frac{\delta}{n_k}) & \sin(\frac{\delta}{n_k}) \\ & & & & -\sin(\frac{\delta}{n_k}) & \cos(\frac{\delta}{n_k}) \end{bmatrix} \begin{bmatrix} \sin(t) \\ \cos(t) \\ \sin(\frac{t}{n_1}) \\ \cos(\frac{t}{n_1}) \\ \dots \\ \dots \\ \sin(\frac{t}{n_k}) \\ \cos(\frac{t}{n_k}) \end{bmatrix}$$

Did you notice that, this equation looks similar to equation just before subsection 4.1, and in fact both are same, just notations are different.

It can be seen that, matrix in the left is independent of t as δ is independent of t . And, multiplication by constant is a linear transformation and hence there are linear relation between two encoding for position t and position $t + \delta$.

5 Conclusion

In summary, positional encoding is a crucial aspect of the transformer architecture that enables it to leverage the sequential nature of data, despite its non-sequential processing capabilities. By using fixed positional encodings based on sine and cosine functions, transformers can maintain an understanding of token positions, while effectively handling various sequence lengths.